

■ Local AI Coding Assistant

Fine-tune · Serve · Chat

Python 3.10+

PyTorch 2.1+

HuggingFace

FastAPI

MIT License

An end-to-end ML engineering project that demonstrates **QLoRA fine-tuning** of a 2B-parameter code model on a consumer 4 GB GPU, served through a unified FastAPI backend with a real-time chat interface.

Technical Documentation · v1.0.0

Table of Contents

1. Project Overview & Features
2. System Architecture
3. Project Structure
4. Engineering Decisions
5. Performance Benchmarks
6. API Reference
7. Quick Start Guide
8. Deployment
9. Limitations & Future Work

1. Project Overview & Features

Local AI Coding Assistant is a fully self-contained, GPU-constrained ML project that demonstrates every layer of the modern AI engineering stack: dataset preparation, parameter-efficient fine-tuning, model quantization, API design, frontend development, benchmarking, and containerised deployment.

Core constraint: the entire training and inference pipeline runs on a single consumer GPU with 4 GB of VRAM (NVIDIA RTX 3050) — no cloud, no multi-GPU cluster.

Key Capabilities

- **QLoRA Fine-Tuning**

Train a 2B-parameter model on 18K Python problems using 4-bit NF4 quantization. Adapter weights occupy only ~1.5 % of total parameters.

- **Unified FastAPI Server**

Single process serves the REST API, Swagger docs, and the web UI — no proxy layers, lower latency, simpler deployment.

- **Chat Interface**

Dark-themed UI with syntax highlighting, one-click code copy, and pre-built suggestion chips for common coding tasks.

- **Benchmarking Suite**

Automated scripts measuring VRAM usage, inference latency, and token throughput, with results persisted to BENCHMARKS.md.

- **Test Suite**

15+ pytest tests with fully mocked model objects — runs in CI without requiring a GPU or downloading model weights.

- **Structured Logging**

Per-request logs capturing HTTP method, path, status code, and duration_ms in a machine-parseable format.

2. System Architecture

The following diagram shows the high-level flow from browser request to model inference. All layers run in a single OS process, eliminating inter-process communication overhead.

Layer	Component	Responsibility
Client	Browser / Chat UI	Sends prompts via <code>fetch()</code> , renders streamed tokens
Transport	HTTP (port 8000)	Single TCP connection, no proxy
Server	FastAPI <code>app.py</code>	Routes, middleware, static file serving
Middleware	<code>LoggingMiddleware</code>	Captures timing, status, path per request
Middleware	<code>CORSMiddleware</code>	Cross-origin policy (tightened in prod)
Model	Quantized 2B (NF4)	~800 MB VRAM at rest, LoRA adapter loaded

Why a Single Server?

The initial prototype used a Flask frontend that proxied requests to a FastAPI backend. This added ~5-15 ms per request, required two processes to manage, and complicated Docker deployments. FastAPI's `StaticFiles` mount and Jinja2 templating support made consolidation straightforward. The result: one uvicorn worker, one port, one Dockerfile.

Request Lifecycle

- **Step 1:** Browser sends `POST /api/generate` with JSON body `{prompt, max_length, temperature}`.
- **Step 2:** `LoggingMiddleware` records start timestamp and request metadata.
- **Step 3:** FastAPI validates the request body against the `GenerateRequest` Pydantic model.
- **Step 4:** The endpoint calls `model_loader.generate()`, which runs tokenization and `model.generate()`.
- **Step 5:** The generated token IDs are decoded, timed, and returned as a `GenerateResponse`.
- **Step 6:** `LoggingMiddleware` appends status code and `duration_ms` to the structured log entry.

3. Project Structure

The repository is organized into five top-level directories, each with a single clear responsibility:

```
local-code-assistant/
├── training/
│   ├── config.py           # All hyperparameters (dataclass)
│   ├── dataset_prep.py     # Download & format dataset
│   └── fine_tune.py        # QLoRA fine-tuning entry point
├── backend/
│   ├── model_loader.py     # Load quantized model + LoRA adapter
│   └── app.py              # Unified FastAPI server (API + UI)
├── frontend/
│   ├── templates/index.html
│   └── static/{css,js}/
├── benchmarks/
│   └── benchmark.py
├── tests/
│   ├── conftest.py         # Pytest fixtures (mocked models)
│   └── test_api.py         # 15+ API tests
├── requirements.txt
├── setup.sh
└── Dockerfile
```

Module Responsibilities

training/config.py

Central dataclass for all hyperparameters — model name, LoRA rank, learning rate, batch size, sequence length. Changing a single field propagates everywhere.

training/dataset_prep.py

Downloads the CodeAlpaca-18K dataset from HuggingFace Hub, applies the instruction-following prompt template, and serialises to disk.

training/fine_tune.py

Entry point for QLoRA training. Loads the base model in 4-bit, wraps it with PEFT, and runs SFTTrainer with gradient checkpointing.

backend/model_loader.py

Loads the fine-tuned LoRA adapter on top of the quantized base model at server startup. Exposes a single `generate()` function.

backend/app.py

FastAPI application with `/api/generate`, `/api/health`, and static file serving. Adds logging and CORS middleware globally.

tests/conftest.py

Replaces `model_loader` with a lightweight mock so all 15+ API tests run in under 10 seconds without GPU or internet access.

4. Engineering Decisions

Every technical choice was driven by one hard constraint: **4 GB of VRAM**. This section explains the rationale behind each decision and the alternatives considered.

4.1 Quantization Format — Why NF4?

Four-bit quantization is necessary to fit a 2B-parameter model in 4 GB alongside the KV cache and optimizer states. Among 4-bit formats, NF4 (Normalized Float 4) outperforms uniform INT4 for neural network weights because weight tensors follow an approximately normal distribution — NF4 maps its 16 quantization levels to match that distribution, minimising rounding error.

Format	Memory (2B params)	Fits in 4 GB?	Notes
FP16	~4,000 MB	No	No headroom for KV cache
INT8	~2,000 MB	Barely	Tight with batch size > 1
NF4	~800 MB	Yes	800 MB leaves plenty of headroom

4.2 Double Quantization — Saving an Extra 100 MB

Standard quantization stores one FP32 scaling factor per block of 64 weights. Double quantization applies a second quantization pass *to those scaling factors*, reducing their storage from FP32 to 8-bit. On a 2B-parameter model this saves approximately 100 MB of VRAM — a meaningful margin on a 4 GB card.

4.3 LoRA Rank Selection

LoRA injects trainable low-rank matrices (rank r) alongside frozen weight matrices. Higher rank captures more task-specific signal but costs more memory. Rank 16 was chosen after empirical testing:

Rank	Trainable Params	Quality	Peak VRAM Impact
4	~4M (0.2%)	Lower	Minimal
8	~8M (0.4%)	Good	Low
16	~30M (1.5%)	Strong (chosen)	Moderate
32	~60M (3.0%)	Slightly better	Higher
64	~120M (6.0%)	Diminishing returns	Much higher

4.4 LoRA Target Modules — `q_proj` and `v_proj`

Transformer attention has four projection matrices: Q, K, V, and O. Following the original LoRA paper, only Q and V are targeted:

- **Q (query):** Controls what the model 'asks' at each position — critical for understanding code structure and intent.
- **V (value):** Controls what information propagates forward — important for generating syntactically and semantically correct code.
- **K and O skipped:** Adding K and O adapters increases memory with marginal quality improvement for code generation tasks specifically.

4.5 Sequence Length — 512 Tokens

KV cache memory scales as $O(\text{seq_len} \times \text{hidden_dim})$. At 512 tokens the cache fits comfortably alongside the model; at 1024+ tokens OOM errors occur with batch size > 1 . Empirically, 512 tokens covers 80%+ of the Python functions in the training corpus, making it an acceptable trade-off.

4.6 Paged AdamW 8-bit Optimizer

Standard AdamW stores momentum and variance tensors per parameter, effectively tripling the memory footprint during training. Paged AdamW 8-bit addresses this in two ways: it halves optimizer state memory by using 8-bit storage, and it pages optimizer states to CPU RAM when GPU memory is full. Combined, this saves approximately 30% of total VRAM compared to FP32 AdamW — the difference between finishing a training run and running out of memory.

5. Performance Benchmarks

All measurements were taken on an NVIDIA RTX 3050 (4 GB VRAM) with an Intel i5/i7 host CPU. Run the benchmark suite yourself with:

```
python -m benchmarks.benchmark
```

5.1 VRAM Usage

Measurement Point	VRAM	Notes
Model loaded (idle)	~800 MB	Base model + LoRA adapter, no active inference
Peak during inference	~1,400 MB	Includes KV cache for 512-token sequence
Peak during training	~3,200 MB	Optimizer states, gradients, activations
OS + CUDA overhead	~400 MB	Baseline before model load

5.2 Inference Performance

Metric	GPU (RTX 3050)	CPU (i5/i7)
Latency (short prompt)	3–5 seconds	25–40 seconds
Latency (long prompt)	5–9 seconds	60–120 seconds
Throughput	25–40 tok/s	2–5 tok/s
First-token latency	~0.8 s	~4 s

GPU inference is 8–10× faster than CPU inference for this model size. The latency delta makes GPU a hard requirement for interactive use.

6. API Reference

POST /api/generate

The primary endpoint. Accepts a coding prompt and generation parameters, returns generated code along with timing metadata.

Request Body

```
{
  "prompt":      "Write a function to reverse a linked list",
  "max_length":  256,
  "temperature": 0.7
}
```

Response Body

```
{
  "generated_code":  "def reverse(head): ...",
  "prompt":          "Write a function to reverse a linked list",
  "model_name":      "codegen-finetuned",
  "generation_time_ms": 3842.1
}
```

Field	Type	Description
prompt	string	The natural-language coding instruction
max_length	integer	Maximum tokens to generate (default 256)
temperature	float	Sampling temperature 0.0–2.0 (default 0.7)

GET /api/health

Health-check endpoint. Returns model load status and server version.

```
{
  "status":      "healthy",
  "model_loaded": true,
  "version":     "1.0.0"
}
```

GET /api/docs

Auto-generated interactive Swagger UI. Explore and test all endpoints directly in the browser at <http://localhost:8000/api/docs>.

Error Handling

All endpoints return standard HTTP status codes. Validation errors from Pydantic produce 422 Unprocessable Entity with a detailed JSON body describing which fields failed and why. Server-side inference errors return 500 Internal Server Error with a structured error message.

7. Quick Start Guide

Prerequisites

- Python 3.10 or newer
- NVIDIA GPU with at least 4 GB VRAM (training) / 2 GB (inference only)
- CUDA toolkit matching your PyTorch version
- Git

Step 1 — Clone and Set Up

```
git clone https://github.com/YOUR_USERNAME/local-code-assistant.git
cd local-code-assistant
chmod +x setup.sh && ./setup.sh
```

setup.sh creates a Python virtual environment, installs all dependencies from requirements.txt, and verifies CUDA availability.

Step 2 — Fine-Tune the Model

```
source .venv/bin/activate
python -m training.fine_tune
```

Training takes 2–3 hours on an RTX 3050. The LoRA adapter is saved to models/codegen-finetuned/. Monitor VRAM usage with nvidia-smi in a second terminal.

Step 3 — Start the Server

```
uvicorn backend.app:app --host 0.0.0.0 --port 8000
```

The server loads the quantized model and adapter, then listens on port 8000. Startup typically takes 15–30 seconds while the model is loaded into VRAM.

Step 4 — Open the Chat UI

Navigate to **http://localhost:8000** in your browser. Type a coding prompt or select one of the pre-built suggestion chips to get started immediately.

Step 5 — Run the Test Suite

```
pytest tests/ -v
```

All tests use mocked models and complete in under 10 seconds with no GPU required. Tests cover prompt validation, response schema, health endpoint, error handling, and CORS headers.

8. Deployment

Local (Development)

```
uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

The `--reload` flag enables hot-reloading of Python source files, useful during development. Do not use `--reload` in production.

Docker

A Dockerfile is included at the repository root. Build and run:

```
docker build -t local-code-assistant .  
docker run --gpus all -p 8000:8000 local-code-assistant
```

The `--gpus all` flag passes all available NVIDIA GPUs into the container. Requires `nvidia-container-toolkit` installed on the host.

HuggingFace Spaces

The project ships a Dockerfile compatible with HuggingFace Spaces (Docker SDK). To deploy:

- Create a new Space at huggingface.co/spaces and select the **Docker** SDK.
- Add your fine-tuned adapter weights to the repository or reference them from the Hub.

```
git remote add space https://huggingface.co/spaces/YOUR_USERNAME/local-code-assistant  
git push space main
```

Note: HuggingFace Spaces free tier provides CPU-only inference. Expect 25–40 second latency per request without a GPU-enabled Space tier.

9. Limitations & Future Work

Engineering maturity includes being transparent about what a system *cannot* do. The following limitations are intentional results of the 4 GB constraint and the project's portfolio scope.

Current Limitations

- **512-Token Context Window**

The model processes at most 512 tokens per request. It cannot reason about multi-file projects, long function chains, or codebases. Complex problems requiring more than ~30 lines of context may produce incomplete or incorrect results.

- **2B Parameter Model Capacity**

A 2B model is orders of magnitude smaller than GPT-4 (~1.8T estimated) or CodeGemma-7B. It may produce syntactically correct but semantically flawed code, and struggles with nuanced multi-step instructions.

- **Quantization Precision Loss**

4-bit NF4 quantization reduces the model footprint by ~5× but introduces precision loss. Output quality is slightly lower than FP16 inference, and numerical instability can occur with very long generation sequences.

- **Single-User Concurrency**

There is no request batching or async inference queue. Only one request is served at a time; concurrent users will experience queuing delays.

- **Python-Only Dataset**

The training corpus is 18K Python problems. The model is not fine-tuned for other languages, web development patterns, or data science workflows.

- **No Production Hardening**

CORS allows all origins, there is no authentication, rate limiting, or input sanitization beyond Pydantic schema validation. Not suitable for public deployment without additional hardening.

Potential Future Work

- Extend context to 1024 tokens using flash-attention-2 for memory-efficient attention.
- Add streaming token output via Server-Sent Events for lower perceived latency.
- Implement an async inference queue (asyncio + semaphore) for multi-user support.
- Evaluate on HumanEval benchmark to produce a quantitative pass@k metric.
- Add multi-language support by fine-tuning on The Stack (multi-language corpus).
- Integrate vLLM or llama.cpp for production-grade inference throughput.

Built with QLoRA · FastAPI · HuggingFace Transformers · bitsandbytes
MIT License · github.com/YOUR_USERNAME/local-code-assistant